

K.Shamela

Final Project

Project Title : Bank-Term Deposit Subscription Prediction

Course : Data Science

Batch : DS-C-WD-E-B144

Bank-Term Deposit Subscription Prediction

Project Overview:

The objective of this project is to predict whether a bank customer will subscribe to a term deposit based on demographic information, financial details, and previous marketing campaign interactions.

The dataset used in this project is the Bank Marketing Dataset, which contains customer information collected during direct marketing campaigns conducted by a Portuguese banking institution.

Since only a small percentage of customers subscribe to a term deposit, the dataset exhibits class imbalance. Therefore, special attention was given to handling the minority class while maintaining overall model performance.

Business Problem:

Banks spend significant resources conducting telemarketing campaigns. Contacting every customer is expensive and time-consuming.

The goal of this project is to develop a predictive model that identifies customers who are more likely to subscribe to a term deposit. This enables the bank to:

- Improve marketing efficiency
- Reduce operational costs
- Increase campaign conversion rates
- Target high-potential customers

Project Architecture Diagram:

Bank Marketing Dataset



Data Cleaning & Encoding



Feature Engineering

(Age×Balance,
Duration×Balance,
Age×Education,
Duration×Education)



Train-Test Split



Model Training

(Logistic Regression,
Random Forest,
XGBoost,
Gradient Boosting,
Naive Bayes,
SVM,
CatBoost,
LightGBM)



Advanced Techniques

(SMOTE,
Stacking,
Blending,
SHAP,
Stratified K-Fold)



Model Evaluation

(Accuracy,
Precision,
Recall,
F1,
ROC-AUC)



Best Model Selection

(LightGBM)



Model Deployment

(Streamlit Dashboard)

Technology Stack:

Programming Language:

- Python

Libraries:

- Pandas
- NumPy
- Scikit-Learn
- XGBoost
- LightGBM
- CatBoost
- SHAP
- Streamlit

- Joblib

Development Environment:

- VS Code
- Jupyter Notebook

Dataset Description:

The dataset contains customer demographic information, account information, and campaign-related attributes.

Important Features

- Age
- Job
- Marital Status
- Education
- Balance
- Housing Loan
- Personal Loan
- Contact Type
- Month of Contact
- Call Duration
- Number of Campaign Contacts
- Previous Campaign Outcome

Dataset:

<https://archive.ics.uci.edu/dataset/222/bank+marketing>

Total Records: 45,211

Features Used: 44

Target Variable

y

- Yes → Customer subscribed
- No → Customer did not subscribe

Data Preprocessing:

Several preprocessing steps were performed before model training.

Data Inspection

- Checked dataset structure
- Verified data types
- Checked for missing values
- Performed exploratory analysis

Missing Value Analysis

No missing values were found in the dataset.

Target Encoding

The target variable was encoded using LabelEncoder:

- No → 0
- Yes → 1

Ordinal Encoding:

The education column was converted into ordinal numerical values:

<u>Education Level</u>		<u>Encoded Value</u>
Unknown	→	0
Primary	→	1
Secondary	→	2
Tertiary	→	3

Categorical Encoding

One-Hot Encoding was applied to categorical variables using `pd.get_dummies()`.

Features encoded:

- Job
- Marital Status
- Default
- Housing Loan
- Personal Loan
- Contact Type
- Month
- Previous Campaign Outcome

This transformation allows machine learning algorithms to process categorical information effectively.

Feature Scaling

StandardScaler was applied to numerical features for models sensitive to feature scale such as:

- Logistic Regression
- Support Vector Machine
- Deep Learning

Feature Engineering:

To improve predictive performance, additional interaction features were created.

Engineered Features

1. age_balance

Customer Age × Account Balance

2. duration_balance

Call Duration × Account Balance

3. age_education

Customer Age × Education Level

4. duration_education

Call Duration × Education Level

These features helped capture hidden relationships within the dataset.

Class Imbalance Handling:

The dataset was highly imbalanced.

Original Distribution

- Class 0 (No Subscription): 31,937
- Class 1 (Subscription): 4,231

Percentage Distribution:

- No Subscription $\approx 88\%$
- Subscription $\approx 12\%$

An imbalanced dataset can cause machine learning models to become biased toward the majority class.

Therefore, special techniques were implemented to address this challenge.

Several imbalance handling techniques were evaluated:

Class Weight Balancing

Applied to:

- Logistic Regression
- Random Forest
- XGBoost

Class weights penalize misclassification of minority class observations more heavily.

SMOTE (Synthetic Minority Oversampling Technique)

Applied to

- LightGBM.

SMOTE generated synthetic samples for the minority class to balance the training dataset.

Training Distribution Before SMOTE:

- Class 0 = 31,937
- Class 1 = 4,231

Training Distribution After SMOTE:

- Class 0 = 31,937
- Class 1 = 31,937

This produced a balanced training dataset.

Threshold Optimization

For LightGBM + SMOTE, probability threshold tuning was performed to maximize the F1-score of the minority class.

Data Leakage Prevention:

To prevent data leakage, the dataset was first divided into training and testing sets using stratified sampling.

SMOTE was applied only to the training dataset after the split.

The testing dataset remained untouched throughout model training and evaluation.

This ensures that model performance accurately reflects real-world deployment conditions.

Model Development:

Multiple machine learning algorithms were trained and evaluated to identify the most effective model for predicting term deposit subscriptions. Each algorithm was selected based on its ability to handle structured tabular data, capture complex relationships, and provide reliable predictions on an imbalanced classification problem.

- Logistic Regression – Baseline linear model.
- Random Forest – Ensemble bagging algorithm.
- Gradient Boosting – Traditional boosting algorithm.
- XGBoost – Advanced gradient boosting.
- LightGBM – Fast and scalable boosting framework.
- CatBoost – Optimized for categorical data.
- SVM – Margin-based classifier.

- Naive Bayes – Probabilistic benchmark model.
- Stacking – Meta-ensemble model.
- Blending – Weighted ensemble model.

Logistic Regression:

Logistic Regression was used as the baseline model for this project. It is a statistical classification algorithm that estimates the probability of a customer subscribing to a term deposit using the logistic (sigmoid) function. The model assumes a linear relationship between the input features and the log-odds of the target variable.

In this project, Logistic Regression provided a benchmark against which more advanced machine learning models could be compared. Class weights were incorporated to reduce the impact of class imbalance and improve minority-class detection.

Advantages:

- Fast training and prediction
- Easy to interpret coefficients
- Provides probability estimates
- Serves as a strong baseline model

Limitations:

- Assumes linear relationships
- May struggle with complex feature interactions

```

Accuracy: 0.9019130819418335

Classification Report:

```

	precision	recall	f1-score	support
0	0.92	0.97	0.95	7985
1	0.65	0.35	0.46	1058
accuracy			0.90	9043
macro avg	0.78	0.66	0.70	9043
weighted avg	0.89	0.90	0.89	9043

```

ROC-AUC Score:
0.9056215990994456

```

Logistic Regression achieved an accuracy of 90.19% and a ROC-AUC score of 0.9056. The model performed well in identifying non-subscribers, achieving 97% recall for Class 0. However, due to class imbalance, it achieved only 35% recall for subscribers, indicating that many potential subscribers were missed.

Logistic Regression balanced:

Accuracy: 0.845626451398872					
	precision	recall	f1-score	support	
0	0.97	0.85	0.91	7985	
1	0.42	0.81	0.55	1058	
accuracy			0.85	9043	
macro avg	0.69	0.83	0.73	9043	
weighted avg	0.91	0.85	0.87	9043	
0.907917965277523					

This tells the model:

"Subscribers are rare. Pay extra attention to them."

So the model becomes more aggressive in predicting YES.

Normal Logistic Regression is conservative (misses many subscribers).

Balanced Logistic Regression is aggressive (finds many subscribers but creates more false alarms).

After applying `class_weight='balanced'`, recall improved from 35% to 81%, meaning the model identified many more potential subscribers. However, precision and overall accuracy decreased because the model generated more false positive predictions. This demonstrates the trade-off between finding more subscribers and maintaining prediction precision.

Random Forest:

Random Forest is an ensemble learning algorithm that combines multiple decision trees using the bagging technique. Each tree is trained on a random subset of the data and features, and the final prediction is obtained through majority voting.

Random Forest was chosen because it can capture non-linear relationships between customer attributes and subscription behavior. It is also less prone to overfitting than a single decision tree and provides feature importance scores that help identify influential variables.

Advantages:

- Handles non-linear patterns effectively
- Robust against overfitting
- Works well with large datasets
- Provides feature importance rankings

- Handles mixed feature types

Limitations:

- Higher computational cost than linear models
- Less interpretable than Logistic Regression

```

Accuracy: 0.9060046444763906

Classification Report:
              precision    recall  f1-score   support

     0       0.92       0.97       0.95       7985
     1       0.66       0.40       0.50       1058

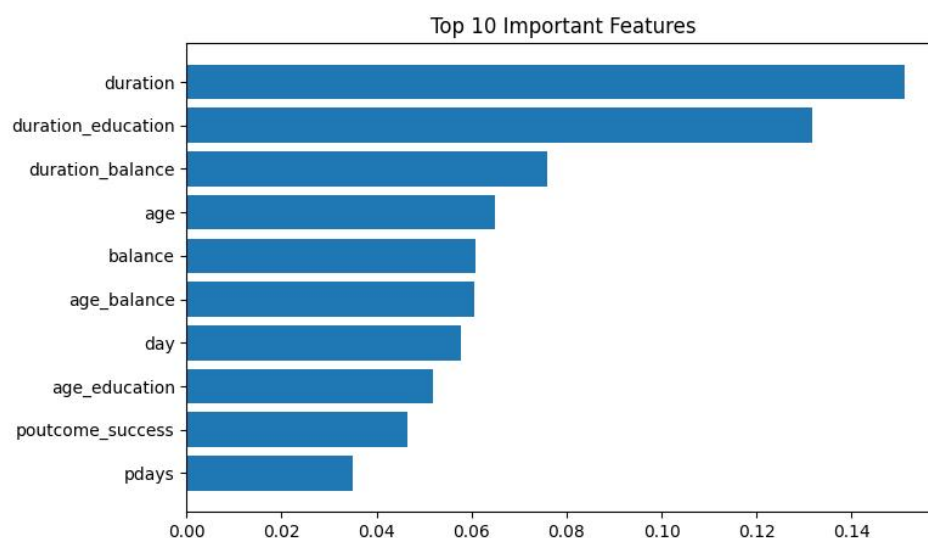
 accuracy          0.91       9043
 macro avg         0.79       0.69       0.72       9043
 weighted avg      0.89       0.91       0.90       9043

ROC-AUC Score:
0.9231854860187994

```

Random Forest achieved an accuracy of 90.60% and a ROC-AUC score of 0.9232. Compared to Logistic Regression, it improved recall from 35% to 40% and increased ROC-AUC significantly. This indicates that Random Forest captured more complex non-linear relationships in the customer data, leading to better discrimination between subscribers and non-subscribers.

Feature Importance:



Random Forest Feature Importance analysis revealed that call duration was the most influential predictor of term deposit subscription. Feature-engineered variables such as duration_education, duration_balance, age_balance, and age_education also ranked among the most important predictors, demonstrating the effectiveness of feature engineering.

Previous campaign success (poutcome_success) and days since last contact (pdays) were also identified as strong predictors of customer subscription behavior.

Random forest Balanced:

Accuracy: 0.9042353201371226					
	precision	recall	f1-score	support	
0	0.92	0.98	0.95	7985	
1	0.68	0.34	0.46	1058	
accuracy			0.90	9043	
macro avg	0.80	0.66	0.70	9043	
weighted avg	0.89	0.90	0.89	9043	
ROC-AUC: 0.9226427623627951					

A Random Forest model with class_weight='balanced' was evaluated to address class imbalance. However, compared to the standard Random Forest model, recall decreased from 40% to 34%, and ROC-AUC decreased slightly from 0.9232 to 0.9226. Therefore, class weighting did not improve Random Forest performance for this dataset and was not selected for further analysis.

XGBoost:

Extreme Gradient Boosting (XGBoost) is an advanced boosting algorithm that builds decision trees sequentially, where each new tree attempts to correct the errors made by previous trees. It incorporates regularization techniques to improve generalization and reduce overfitting.

XGBoost is widely recognized for its strong performance in machine learning competitions and real-world predictive analytics tasks. In this project, it demonstrated excellent predictive capability by capturing complex interactions among customer demographics, financial information, and campaign-related variables.

Advantages:

- High predictive accuracy
- Built-in regularization mechanisms
- Handles missing values efficiently
- Supports feature importance analysis
- Effective for structured tabular data

Limitations:

- Requires hyperparameter tuning

- Computationally intensive compared to simpler models

```

Accuracy: 0.9052305650779608

Classification Report:
      precision    recall  f1-score   support

     0       0.93      0.96      0.95      7985
     1       0.63      0.47      0.54      1058

   accuracy          0.91      9043
  macro avg       0.78      0.72      0.74      9043
 weighted avg       0.90      0.91      0.90      9043

ROC-AUC Score:
0.9276597306149409

```

XGBoost achieved an accuracy of 90.52% and a ROC-AUC score of 0.9277. Compared to Random Forest, it improved recall from 40% to 47%, enabling the model to identify more potential subscribers. XGBoost uses gradient boosting, where each tree learns from the mistakes of previous trees, resulting in stronger predictive performance and better ranking ability.

XGBoost Balanced:

The dataset is imbalanced, with 39,922 non-subscribers and 5,289 subscribers. I calculated the imbalance ratio as $39,922 \div 5,289 \approx 7.5$ and used this value for `scale_pos_weight`. This makes XGBoost pay more attention to the minority class during training and improves recall for subscriber prediction.

```

Accuracy: 0.8764790445648568

Classification Report:
      precision    recall  f1-score   support

     0       0.97      0.89      0.93      7985
     1       0.48      0.80      0.60      1058

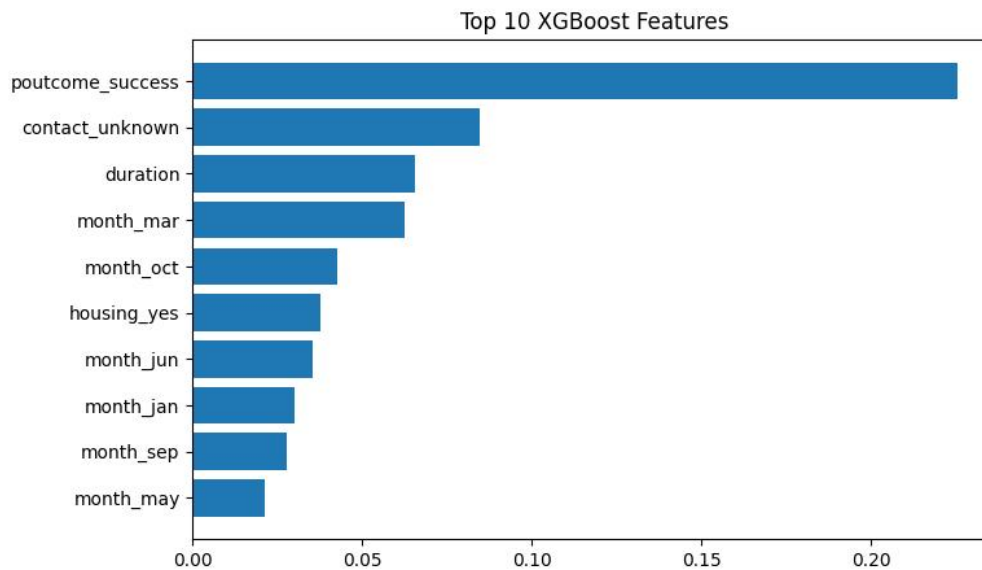
   accuracy          0.88      9043
  macro avg       0.73      0.84      0.77      9043
 weighted avg       0.91      0.88      0.89      9043

ROC-AUC Score:
0.9251726713485705

```

To address class imbalance, a Balanced XGBoost model was trained using `scale_pos_weight=7.5`, which reflects the ratio between majority and minority classes. The balanced model significantly improved recall from 47% to 80%, enabling the detection of substantially more potential subscribers. However, this improvement came with lower precision and slightly reduced ROC-AUC.

Feature Importance:



XGBoost Feature Importance Analysis

XGBoost identified previous campaign success (poutcome_success) as the most influential predictor of term deposit subscription. Contact method (contact_unknown), call duration (duration), and month-related variables also ranked highly. These findings suggest that previous customer engagement and campaign timing play a significant role in predicting subscription behavior. Unlike Random Forest, XGBoost relied less on engineered interaction features and more on campaign-related attributes.

Gradient Boosting:

Gradient Boosting is an ensemble technique that builds weak learners sequentially. Each new model focuses on correcting the residual errors generated by previous models, gradually improving overall predictive performance.

The algorithm was included to evaluate the effectiveness of traditional boosting methods before comparing them with more advanced implementations such as XGBoost, LightGBM, and CatBoost.

Advantages:

- Strong predictive performance
- Captures complex relationships
- Reduces bias through iterative learning
- Effective for classification tasks

Limitations:

- Longer training time

- Sensitive to hyperparameter settings

```

Accuracy: 0.9042353201371226

Classification Report:
              precision    recall  f1-score   support

     0       0.92      0.97      0.95      7985
     1       0.65      0.40      0.49      1058

 accuracy          0.90      9043
  macro avg       0.79      0.69      0.72      9043
 weighted avg     0.89      0.90      0.89      9043

ROC-AUC Score:
0.9216834376364946

```

Gradient Boosting Classifier was evaluated, It achieved an accuracy of 90.42% and a ROC-AUC score of 0.9217. The model improved performance over Logistic Regression but was slightly outperformed by Random Forest and significantly outperformed by advanced boosting algorithms such as XGBoost, CatBoost, and LightGBM. Therefore, it was retained as an intermediate benchmark model rather than a final candidate

Naive Bayes:

Naive Bayes is a probabilistic classification algorithm based on Bayes' Theorem. It assumes that all predictor variables are conditionally independent given the target class.

Although this assumption is rarely true in real-world datasets, Naive Bayes often performs surprisingly well and serves as a useful benchmark model. It was included to compare a probability-based approach against tree-based and boosting algorithms.

Advantages:

- Extremely fast training and prediction
- Computationally efficient
- Performs well on smaller datasets
- Produces probabilistic outputs

Limitations:

- Strong independence assumption
- Limited ability to model complex feature interactions

Accuracy: 0.8709499060046445					
Classification Report:					
	precision	recall	f1-score	support	
0	0.91	0.95	0.93	7985	
1	0.43	0.30	0.35	1058	
accuracy			0.87	9043	
macro avg	0.67	0.62	0.64	9043	
weighted avg	0.85	0.87	0.86	9043	
ROC-AUC Score:					
0.7956569678733638					

It shows GaussianNB struggles to separate subscribers from non-subscribers.

Why did Naive Bayes perform poorly?

Because the Bank Marketing dataset contains many correlated features.

For example:

- Age and balance are related.
- Previous campaign outcome and duration are related.
- Housing loan and balance may be related.

Naive Bayes assumes independence between features, which doesn't hold here.

As a result, its probability estimates are less accurate.

Gaussian Naive Bayes was evaluated as a probabilistic baseline classifier. The model achieved an accuracy of 87.09% and a ROC-AUC score of 0.7957. Due to its assumption that input features are conditionally independent, it was unable to capture the complex relationships present in the Bank Marketing dataset.

Consequently, its performance was substantially lower than ensemble-based models such as Random Forest, XGBoost, CatBoost, and LightGBM, and it was not considered for the final deployment model.

Support Vector Machine (SVM):

Support Vector Machine is a supervised learning algorithm that identifies the optimal hyperplane that maximizes the separation between classes. The algorithm focuses on observations closest to the decision boundary, known as support vectors.

SVM was included because of its strong theoretical foundation and ability to perform well in high-dimensional feature spaces. It can model complex decision boundaries through kernel functions.

Advantages:

- Effective in high-dimensional datasets
- Strong generalization capability
- Robust against overfitting
- Suitable for complex classification problems

Limitations:

- Computationally expensive on large datasets
- Sensitive to parameter selection
- Less scalable than boosting algorithms

```
Accuracy: 0.9015813336282207

Classification Report:
              precision    recall  f1-score   support

     0           0.92       0.98       0.95       7985
     1           0.65       0.34       0.44       1058

 accuracy          0.90       0.90       0.90       9043
 macro avg         0.79       0.66       0.69       9043
weighted avg         0.89       0.90       0.89       9043

ROC-AUC Score:
0.8908704056400647
```

Support Vector Machine (SVM) was trained using standardized features because SVM relies on distance calculations and is sensitive to feature scales. The model achieved an accuracy of 90.16% and a ROC-AUC score of 0.8909. Although its overall accuracy was comparable to Logistic Regression, it achieved lower recall and ROC-AUC than the ensemble methods. Due to the large dataset size, class imbalance, and complex feature relationships, SVM was outperformed by tree-based ensemble models such as XGBoost, CatBoost, and LightGBM.

LightGBM:

Light Gradient Boosting Machine (LightGBM) is a gradient boosting framework developed by Microsoft. Unlike traditional boosting algorithms, LightGBM uses histogram-based learning and leaf-wise tree growth, enabling faster training and improved efficiency.

LightGBM was one of the best-performing models in this project. It effectively captured complex customer behavior patterns while maintaining fast training and prediction speeds. The model also integrated well with SHAP explainability techniques, making it suitable for deployment in a business environment.

Advantages:

- Fast training and inference
- Lower memory consumption
- Excellent predictive performance
- Handles large datasets efficiently
- Supports feature importance and explainability
- Scalable for production deployment

Limitations:

- Can overfit if not properly tuned
- Requires careful hyperparameter optimization

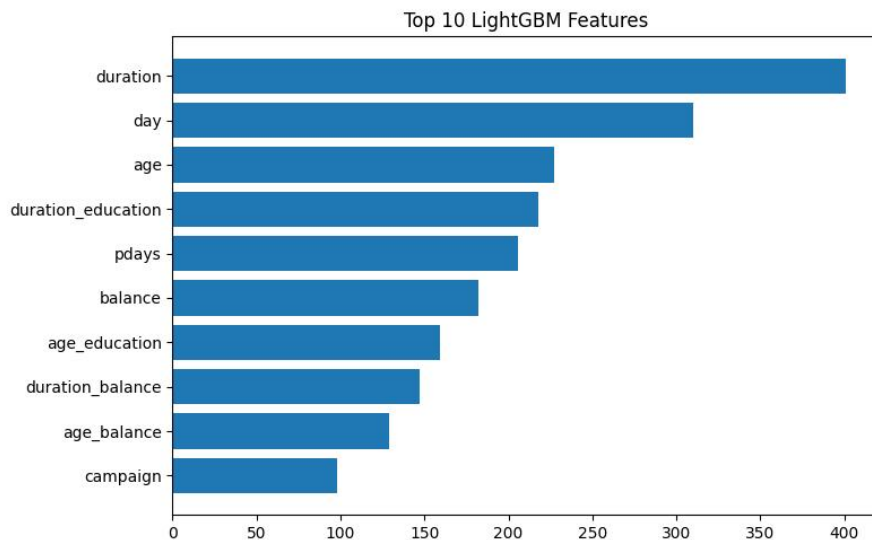
LightGBM was ultimately selected as the final deployment model due to its balance of accuracy, efficiency, scalability, and interpretability.

```
[LightGBM] [Info] Score training from score: 2101521
Accuracy: 0.9089903792989052
```

Classification Report:				
	precision	recall	f1-score	support
0	0.93	0.97	0.95	7985
1	0.65	0.48	0.55	1058
accuracy			0.91	9043
macro avg	0.79	0.72	0.75	9043
weighted avg	0.90	0.91	0.90	9043
ROC-AUC Score:				
			0.9345867073541717	

LightGBM achieved the highest ROC-AUC score (0.9346) among all individual models while maintaining high accuracy and a balanced precision-recall trade-off. It also trains efficiently using a leaf-wise boosting strategy.

LightGBM Feature Importance:



LightGBM identified **call duration (duration)** as the most influential predictor of term deposit subscription. Other important features included **day of contact, customer age, account balance, and previous contact information (pdays)**. In addition, several engineered interaction features (duration_education, age_education, duration_balance, and age_balance) ranked among the top predictors, demonstrating that combining related variables helped the model capture more complex customer behaviour. These findings highlight the effectiveness of feature engineering in improving model performance.

LightGBM +SMOTE:

SMOTE stands for Synthetic Minority Oversampling Technique.

Its purpose is to create new synthetic samples for the minority class ("Yes") instead of simply duplicating existing ones.

Notice:

- Only the training data is balanced.
- The test data remains untouched.

This is very important because modifying the test set would cause **data leakage**.

Class Distribution

Dataset	No (0)	Yes (1)
Before SMOTE	31,937	4,231

Dataset	No (0)	Yes (1)
After SMOTE	31,937	31,937

The test dataset was **not modified** to ensure an unbiased evaluation and prevent data leakage.

I applied SMOTE as an experiment to address the class imbalance in the training data. It successfully increased the Recall from 48% to 87%, meaning the model detected many more actual subscribers. However, this also increased false positives, reducing Accuracy, Precision, and ROC-AUC. Since my project's objective was to achieve the best overall predictive performance, I selected the standard LightGBM model as the final individual model and reported the SMOTE results as a comparative imbalance-handling experiment.

CatBoost:

CatBoost is a gradient boosting algorithm developed by Yandex that is specifically designed to handle categorical variables efficiently. It uses ordered boosting techniques to reduce prediction bias and improve generalization performance.

CatBoost was included because the Bank Marketing dataset contains several categorical features related to customer demographics and campaign information. The algorithm demonstrated strong predictive performance while requiring minimal preprocessing.

Advantages:

- Excellent handling of categorical features
- Reduces overfitting through ordered boosting
- Strong predictive accuracy
- Minimal preprocessing requirements
- Stable performance across different datasets

Limitations:

- Longer training time than LightGBM
- Higher computational requirements for large datasets

```

Accuracy: 0.9095432931549264

Classification Report:
              precision    recall  f1-score   support

     0       0.93       0.97       0.95       7985
     1       0.65       0.48       0.55       1058

   accuracy          0.91       9043
  macro avg       0.79       0.72       0.75       9043
 weighted avg       0.90       0.91       0.90       9043

ROC-AUC Score:
0.9325008019526214

```

CatBoost achieved an **Accuracy of 90.95%, Precision of 0.65, Recall of 0.48, F1-score of 0.55**, and a **ROC-AUC score of 0.9325**. Its performance was comparable to LightGBM, making it one of the strongest individual models evaluated. However, LightGBM achieved a slightly higher ROC-AUC score and was therefore selected as the final individual model.

Deep Learning Neural Network

A feed-forward Artificial Neural Network consisting of:

- Input Layer
- Hidden Layers
- Output Layer

Architecture:

- Dense (64 neurons)
- Dense (32 neurons)
- Dense (16 neurons)
- Output Layer (Sigmoid)

Advantages:

- Learns complex non-linear patterns
- Highly flexible

An Artificial Neural Network (ANN) was developed using TensorFlow's Sequential API. The network consisted of three hidden layers containing **64**, **32**, and **16** neurons with **ReLU** activation functions, followed by a single output neuron with a **Sigmoid** activation function for binary classification. The model was trained using the **Adam**

optimizer and Binary Cross-Entropy loss function for 20 epochs with a batch size of 32. A validation split of 20% was used to monitor performance during training.

```
... Accuracy: 0.9047882339931439
      precision    recall  f1-score   support

      0       0.95      0.95      0.95     7985
      1       0.59      0.59      0.59     1058

   accuracy          0.90     9043
  macro avg       0.77      0.77      0.77     9043
 weighted avg       0.90      0.90      0.90     9043

ROC-AUC: 0.9215939503771842
```

The ANN achieved an **Accuracy of 90.48%, Precision of 0.59, Recall of 0.59, F1-score of 0.59,** and a **ROC-AUC score of 0.9216.**

Cross Validation:

To ensure robustness, Stratified K-Fold Cross Validation was applied.

Configuration:

- 5 folds
- Stratified sampling

To evaluate the robustness of the selected LightGBM model, 5-Fold Stratified Cross Validation was performed using ROC-AUC as the evaluation metric. Stratified K-Fold preserves the original class distribution in every fold, making it suitable for imbalanced classification problems. The model achieved ROC-AUC scores of **0.9403, 0.9365, 0.9347, 0.9378,** and **0.9308,** with an average ROC-AUC of **0.9360** and a standard deviation of **0.0032.** The very low standard deviation indicates that the model produces stable and consistent performance across different data splits, demonstrating good generalization ability.

Results:

Mean ROC-AUC = 0.9360

Standard Deviation = 0.0032

The low standard deviation indicates stable and consistent model performance.

Explainable AI (SHAP)

SHAP (SHapley Additive exPlanations) was used to interpret model decisions.

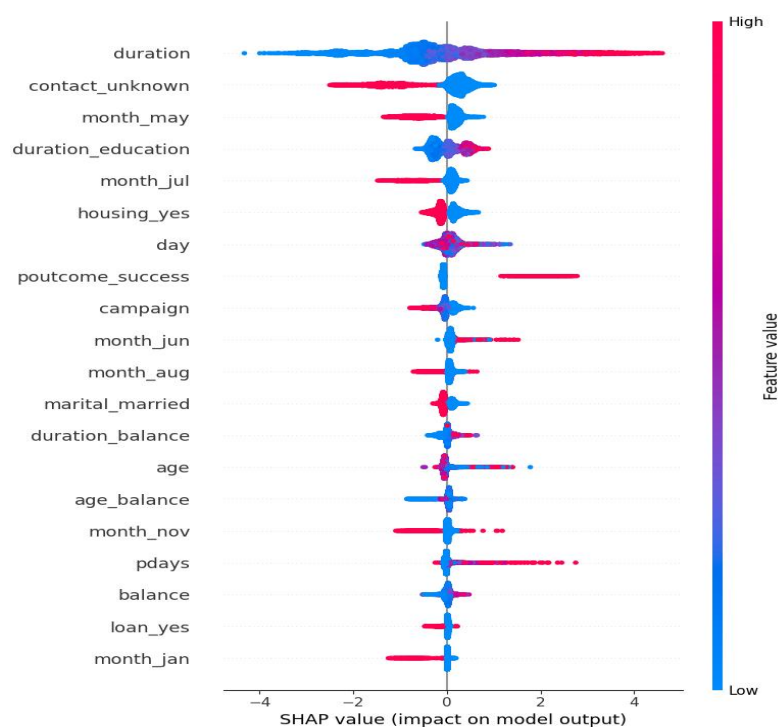
Benefits:

- Identifies most influential features
- Improves model transparency
- Supports business decision-making

Key Influential Features:

- Duration
- Balance
- Previous Campaign Outcome
- Contact Month
- Campaign Count

SHAP analysis provided insights into why predictions were made.



Y-axis

The left side lists the features.

The features are ordered from **most important** to **least important** based on their average SHAP impact.

X-axis

The horizontal axis is:

SHAP Value

It measures the effect of a feature on the model's prediction.

- Positive SHAP value → pushes the prediction toward **"Yes"** (subscription).
- Negative SHAP value → pushes the prediction toward **"No"** (no subscription).

Colors

The color represents the actual feature value.

-  Blue = Low feature value
-  Red = High feature value

Examples:

- For **duration**:
 - Blue = short call duration
 - Red = long call duration

Interpreting Your Features

1. duration

This is the top feature.

Notice:

- Red points (high duration) are mostly on the **right**.
- Blue points (low duration) are mostly on the **left**.

Interpretation:

- Longer calls increase the probability of subscription.
- Short calls decrease the probability.

This matches business intuition: customers who stay on the phone longer are usually more interested.

2. contact_unknown

High values (red) are mostly on the **left**.

Interpretation:

- When the contact type is unknown, the prediction shifts toward **No**.

- Known contact methods have a more positive effect.

3. month_may

Most red points are on the left.

Interpretation:

- Campaigns conducted in May tend to reduce the probability of subscription in this dataset.

4. duration_education

High values appear more on the right.

Interpretation:

- The interaction between longer call duration and higher education generally increases the likelihood of subscription.

5. housing_yes

Red points are mainly on the left.

Interpretation:

- Customers with a housing loan are slightly less likely to subscribe.

6. poutcome_success

Almost all red points are on the right.

Interpretation:

- A successful outcome in a previous campaign strongly increases the probability of subscribing again.
- This is one of the strongest positive indicators.

7. campaign

Red points are mostly on the left.

Interpretation:

- A higher number of campaign contacts tends to reduce the probability of subscription.
- Repeatedly contacting the same customer may indicate lower interest.

8. pdays

Many red points appear on the right.

Interpretation:

- Larger pdays values (days since previous contact, depending on dataset encoding) contribute positively for many customers in this trained model. This relationship comes from the learned data pattern and should be interpreted together with the dataset's definition of pdays.

Why are there so many dots?

Each dot represents **one customer**.

Since your test set contains:

9043 customers

Each row contains approximately

9043 dots

The spread of the dots shows how differently that feature affects different customers.

Ensemble Learning:

Stacking:

Stacking is an **ensemble learning technique**.

Instead of relying on **one model**, stacking combines the predictions of **multiple models** to make a final prediction.

Think of it like a panel of experts.

Example

Imagine predicting whether a customer will subscribe.

You ask three experts:

- Expert 1 → Random Forest
- Expert 2 → XGBoost
- Expert 3 → LightGBM

Each expert gives their opinion.

Then a **fourth expert** looks at those opinions and makes the final decision.

That fourth expert is called the **Meta Learner**.

Base Models:

- Random Forest
- XGBoost
- LightGBM

Meta Learner:

- Logistic Regression

Model Configuration

Component	Algorithm
Base Model 1	Random Forest
Base Model 2	XGBoost
Base Model 3	LightGBM
Meta Learner	Logistic Regression

Performance

Metric	Value
Accuracy	91.02%
Precision	0.66
Recall	0.48

Metric	Value
F1-Score	0.56
ROC-AUC	0.9347

```

Accuracy: 0.910206789782152

Classification Report:
              precision    recall  f1-score   support

     0           0.93       0.97       0.95       7985
     1           0.66       0.48       0.56       1058

   accuracy          0.91       0.91       0.91       9043
  macro avg           0.80       0.72       0.75       9043
 weighted avg          0.90       0.91       0.90       9043


ROC-AUC Score:
0.9347085094571225

```

ROC-AUC:

0.9347

Purpose:

Combine strengths of multiple algorithms.

Advantages:

- Combines strengths of multiple models
- Often achieves superior predictive performance

Blending

Blending is an ensemble learning technique that combines the prediction probabilities of multiple trained models using predefined weights. Unlike stacking, it does not train a meta learner.

Weighted Average:

- Random Forest = 20%

- XGBoost = 30%
- LightGBM = 50%

```

Accuracy: 0.9097644586973349

Classification Report:

```

	precision	recall	f1-score	support
0	0.93	0.97	0.95	7985
1	0.66	0.47	0.55	1058
accuracy			0.91	9043
macro avg	0.80	0.72	0.75	9043
weighted avg	0.90	0.91	0.90	9043

```

ROC-AUC:
0.9352588087541267

```

The Blending Ensemble achieved the highest ROC-AUC score among all evaluated models, indicating the strongest ability to distinguish between customers who subscribed and those who did not. By combining the strengths of three high-performing classifiers through weighted averaging, the ensemble produced a robust and reliable prediction model.

ROC-AUC:

0.9353

Purpose:

Combine probability outputs from strong individual models.

Model Evaluation Metrics:

To evaluate the performance of the machine learning models, multiple evaluation metrics were used instead of relying solely on accuracy. Since the Bank Marketing dataset is imbalanced, a model can achieve high accuracy by predicting the majority class most of the time while failing to identify potential subscribers. Therefore, additional metrics such as Precision, Recall, F1-Score, ROC-AUC, and PR-AUC were considered to obtain a comprehensive assessment of model performance.

Accuracy

Accuracy measures the overall proportion of correctly classified observations.

Formula:

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

Where:

- TP = True Positives
- TN = True Negatives
- FP = False Positives
- FN = False Negatives

Interpretation:

A high accuracy indicates that the model correctly predicts a large percentage of observations. However, in imbalanced datasets, accuracy alone can be misleading because the model may simply predict the majority class and still achieve a high score.

Precision

Precision measures the proportion of customers predicted as subscribers who actually subscribed.

Formula:

$$\text{Precision} = TP / (TP + FP)$$

Interpretation:

High precision means that when the model predicts a customer will subscribe, it is usually correct. This metric is important when the cost of contacting uninterested customers is high, as it helps reduce unnecessary marketing efforts.

Recall

Recall measures the proportion of actual subscribers that were correctly identified by the model.

Formula:

$$\text{Recall} = TP / (TP + FN)$$

Interpretation:

High recall indicates that the model successfully identifies most potential subscribers. In marketing campaigns, recall is important because missing a potential subscriber may result in lost business opportunities.

F1-Score

The F1-Score is the harmonic mean of Precision and Recall.

Formula:

$$F1\text{-Score} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

Interpretation:

The F1-Score provides a balanced measure when both Precision and Recall are important. It is particularly useful for imbalanced datasets because it considers both false positives and false negatives.

ROC-AUC (Receiver Operating Characteristic – Area Under Curve)

ROC-AUC evaluates the model's ability to distinguish between subscribers and non-subscribers across different classification thresholds.

The ROC curve plots:

- True Positive Rate (Recall)
- False Positive Rate

Interpretation:

A higher ROC-AUC value indicates better discrimination between classes.

ROC-AUC Values:

- 0.50 = Random guessing
- 0.70–0.80 = Acceptable performance
- 0.80–0.90 = Good performance
- Above 0.90 = Excellent performance

ROC-AUC was selected as the primary ranking metric because it evaluates model performance across all classification thresholds and is less sensitive to a specific decision boundary.

Why Multiple Metrics Were Used

No single metric can fully describe model performance. Therefore, multiple evaluation metrics were used to assess different aspects of the models:

- Accuracy evaluates overall correctness.
- Precision measures prediction reliability.
- Recall measures the ability to identify subscribers.
- F1-Score balances Precision and Recall.
- ROC-AUC evaluates discrimination capability across thresholds.

Using multiple metrics ensured that the selected model was not only accurate but also effective at identifying potential term deposit subscribers, which is the primary business objective of this project.

Model Comparison:

The performance of all trained models was evaluated using Accuracy, Precision, Recall, F1-Score, and ROC-AUC. Since the dataset is imbalanced, ROC-AUC was considered the primary metric for ranking the models.

Rank	Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
1	Blending	0.9098	0.66	0.47	0.55	0.9353
2	Stacking	0.9102	0.66	0.48	0.56	0.9347
3	LightGBM	0.9090	0.65	0.48	0.55	0.9346
4	CatBoost	0.9095	0.65	0.48	0.55	0.9325
5	XGBoost	0.9052	0.63	0.47	0.54	0.9277
6	LightGBM + SMOTE	0.8474	0.43	0.87	0.57	0.9238
7	Random Forest	0.9060	0.66	0.40	0.50	0.9232
8	Gradient Boosting	0.9042	0.65	0.40	0.49	0.9217
9	Logistic Regression	0.9019	0.65	0.35	0.46	0.9056
10	SVM	0.9016	0.65	0.34	0.44	0.8909
11	Naive Bayes	0.8709	0.43	0.30	0.35	0.7957

Observation:

The Blending ensemble achieved the highest ROC-AUC score of 0.9353, indicating the strongest overall discrimination capability among all evaluated models. Stacking and LightGBM followed closely with ROC-AUC scores of 0.9347 and 0.9346 respectively, demonstrating highly competitive performance.

LightGBM combined with SMOTE achieved the highest Recall (0.87), showing excellent ability to identify potential subscribers. However, this improvement came at the cost of lower Precision and Accuracy, resulting in more false positive predictions.

Although Blending marginally outperformed all other models, LightGBM was selected as the final deployment model because it delivered nearly identical predictive performance while offering faster training, lower computational requirements, easier maintenance, and better suitability for real-time deployment. Therefore, LightGBM provided the best balance between performance, scalability, and operational efficiency.

Imbalance Handling Experiments:

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	0.9019	0.65	0.35	0.46	0.9056
Logistic Regression Balanced	0.8500	0.42	0.81	0.55	0.9079
Random Forest	0.9060	0.66	0.40	0.50	0.9232
Random Forest Balanced	0.9042	0.68	0.34	0.46	0.9226
XGBoost	0.9052	0.63	0.47	0.54	0.9277
XGBoost Balanced	0.8765	0.48	0.80	0.60	0.9252
LightGBM	0.9090	0.65	0.48	0.55	0.9346
LightGBM + SMOTE	0.8474	0.43	0.87	0.57	0.9238

The dataset exhibited significant class imbalance, with only about 11.7% positive subscription cases. Multiple imbalance handling strategies were evaluated, including class weighting and SMOTE. Balanced Logistic Regression and Balanced XGBoost substantially improved minority-class recall from 0.35 to 0.81 and from 0.47 to 0.80 respectively. LightGBM combined with SMOTE achieved the highest recall (0.87), indicating strong detection of positive subscription cases. However, this improvement came at the cost of reduced accuracy and precision. Therefore, the original LightGBM model was selected as the final deployment model because it provided the best overall balance between Accuracy (90.90%), F1-Score (0.55), and ROC-AUC (0.9346).

Model Selection:

Although the Blending model achieved the highest ROC-AUC score (0.9353), the improvement over LightGBM (0.9346) was very small (0.0007). LightGBM was selected as the final deployment model because it provides simpler deployment, faster prediction, lower complexity, easier maintenance, and better interpretability while maintaining nearly identical predictive performance.

Then compare:

Criterion	LightGBM	Blending
ROC-AUC	0.9346	0.9353
Training Complexity	Low	Medium
Deployment Complexity	Easy	Moderate
Inference Speed	Fast	Slower
Explainability	High	Moderate
Model Maintenance	Easy	More Complex

Reasons:

1. Nearly identical performance to Stacking
2. Simpler architecture
3. Faster inference time
4. Easier deployment
5. Lower maintenance complexity
6. Strong explainability support

Therefore, LightGBM provided the best balance between performance and practicality.

LightGBM

Accuracy: 90.90%

ROC-AUC: 93.46%

Cross-Validation ROC-AUC: 93.60%

Deployment:

The selected LightGBM model was deployed using Streamlit.

The application allows users to:

- Enter customer information
- Generate subscription predictions
- View subscription probability

Deployment Components:

- Streamlit Dashboard
- LightGBM Model
- StandardScaler
- Feature Column Mapping

Business Insights:

Key findings include:

- Call duration strongly influences subscription likelihood.
- Previous successful campaign outcomes increase conversion probability.
- Customer balance contributes significantly to predictions.
- Certain contact months yield higher subscription rates.

These insights can help banks optimize future marketing campaigns.

Future Scope:

- Hyperparameter optimization using Optuna.
- Real-time deployment using cloud platforms.
- Integration with CRM systems.
- Automated customer segmentation.
- Explainable AI dashboards.
- Deep learning optimization using TabNet or Transformer models.

Conclusion:

This project successfully developed a machine learning solution for predicting bank term deposit subscriptions.

Multiple machine learning, ensemble learning, and deep learning models were evaluated. LightGBM demonstrated strong predictive performance and was selected for deployment after extensive evaluation, explainability analysis, imbalance handling, and cross-validation.

The final solution can assist banks in identifying high-potential customers, improving campaign effectiveness, and reducing marketing costs.